# HAFTA9 LAB DERSİ

## Özet

Önceki hafta (hafta8)  ara sınav oldunuz. Ara sınav öncesi çarşamba derste işlenenler:

- Değer döndüren - döndürmeyen - parametreli - parametresiz fonksiyonlar
- Recursive fonksiyon
- Tip değişimi

**Sınavla İlgili Değerlendirmeler**

Yanlış x%10=y;  doğru: y=x%10; // atama operatorü sağa yazılmaz.
Öperator önceliği konusuna yeniden bakın.
return sayi; //eğer değer donduruyorsan return ile bir şey donmeli unutulmuş sınavda.
Global local değişken ayrımında takılmışsınız.

## HATIRLAYALIM

**Değer dondurmek**

Fonksiyon return ile bitecek fonksiyon int double vs ile başlar return sayi

**Deger dondurmemek**

return sayı; bir ifade yok  fonksiyon void ile başlar.

**Parametre almak**

() yay parantez içinde  (int x,int y) gibi

**Parametre almamak**

() () yay parantez içinde bir şey yazmaz ya da (void) olur.

**prototype**

int fonk(void); // mainden sonra yazılmışsa gerekir.

**Nasıl çağrırırız?**

**Değer döndüren fonksiyon**

Ya bir şeye atanır ya da printf içinde çağrılır

```
x= fonk(2,3);
printf(“Toplam: %d”, fonk(2,3));
```

**Değer döndürmeyen fonksiyon**

Direk çağrılır.

```
fonk(2,3);          fonk();
```

**Reqursive (Özyinemeli) Fonksiyon**

Bir problemin daha küçük ve benzer alt problemlere bölünerek, fonksiyonun kendi kendini çağırması yoluyla çözüldüğü bir programlama tekniğidir**.** Bir rekürsif fonksiyonun düzgün çalışabilmesi için iki temel yapıya ihtiyacı vardır:

**Taban Durumu (Base Case / Sınır):** Fonksiyonun kendini çağırmayı durdurduğu ve doğrudan bir sonuç döndürdüğü en küçük adımdır. Bu durum olmazsa fonksiyon sonsuza kadar çalışır ve sistem hafızası dolarak hata verir.

**Özyineli Durum (Recursive Case):** Problemin küçültülerek fonksiyonun tekrar çağrıldığı kısımdır. Her çağrıda problem taban duruma bir adım daha yaklaşmalıdır.

```
#include <stdio.h>

int faktoriyel(int n){
    // Negatif sayı kontrolü
    if (n < 0) {
        printf("Hata: Negatif sayıların faktöriyeli yoktur!\n");
        return -1; // Hata kodu
    }

    // Taban durumu (0 ve 1 için faktöriyel 1'dir)
    if (n == 0 || n == 1)
        return 1;

    // Özyineli durum
    return n * faktoriyel(n - 1);
}

int main()
{
    int a;
    printf("sayi gir\n");
    scanf("%d", &a);

    int sonuc = faktoriyel(a);
    if (sonuc != -1) {
        printf("Sonuc: %d\n", sonuc);
    }

    return 0;
}
```

```
#include <stdio.h>

int faktoriyel(int n) {
    // Negatif sayı kontrolü
    if (n < 0) {
```

```
        printf("Hata: Negatif sayıların faktöriyeli yoktur!\n");
        return -1; // Hata kodu
    }

    int sonuc = 1;

    // 0! ve 1! için döngüye girmez, sonuç direkt 1 kalır
    for (int i = 2; i <= n; i++) {
        sonuc *= i; // sonuc = sonuc * i;
    }

    return sonuc;
}

int main() {
    int a;
    printf("sayi gir\n");
    scanf("%d", &a);

    int sonuc = faktoriyel(a);
    if (sonuc != -1) {
        printf("Sonuc: %d\n", sonuc);
    }

    return 0;
}
```

# TİP DÖNÜŞÜMÜ

### 1. Otomatik (Örtülü) Tip Dönüşümü (Implicit Type Conversion)

Derleyici, veri kaybı olmayacak veya işlem sırasında uyumluluğu sağlamak için türleri otomatik olarak dönüştürür. Genellikle küçük veri tipleri daha büyük veri tiplerine dönüştürülür (örneğin int → float).

```
#include <stdio.h>
int main() {
    int a = 5;
    float b = 2.0;

    // 'a' değişkeni otomatik olarak float türüne (5.0) dönüştürülür
    float sonuc = a + b;

    printf("Sonuc: %.2f\n", sonuc); // Çıktı: 7.00
    return 0;
}
```

### 2. Bilinçli (Açık) Tip Dönüşümü (Explicit Type Casting)

Programcının kendi isteğiyle, parantez içinde hedef veri tipini belirterek yaptığı dönüştürme işlemidir. Özellikle tamsayı bölmelerinde ondalıklı sonuç almak için sıkça kullanılır.

Sözdizimi (Syntax):

```
(yeni_veri_tipi) değişken_veya_değer;

#include <stdio.h>
int main() {
    int toplam = 9;
    int kisi = 2;

    // Hatalı yaklaşım: int / int işlemi sonuç olarak tamsayı (4) verir.
    // float sonuc = toplam / kisi; -> 4.00 olur.

    // Bilinçli tip değişimi (Casting):
    float ortalama = (float)toplam / kisi;

    printf("Ortalama: %.2f\n", ortalama); // Çıktı: 4.50
    return 0;
}
```

**C dilinde Global (Küresel) ve Local (Yerel) değişkenler, değişkenlerin nerede tanımlandığına ve programın hangi kısımlarından erişilebileceğine (kapsam - scope) göre ayrılır.**

## Temel Farklar Nelerdir?

| Özellik | Yerel (Local) Değişken | Küresel (Global) Değişken |
|---|---|---|
| Tanımlandığı Yer | Bir fonksiyonun veya blok { } içinde. | Tüm fonksiyonların dışında (genellikle dosyanın en üstünde). |
| Erişilebilirlik | Sadece tanımlandığı fonksiyon veya blok içinden erişilebilir. | Programdaki tüm fonksiyonlar tarafından erişilebilir. |
| Yaşam Süresi | Fonksiyon çağrıldığında oluşturulur, fonksiyon bittiğinde bellekten silinir. | Program çalışmaya başladığı an belleğe yüklenir, program sonlanana kadar hafızada kalır. |

```
#include <stdio.h>
```

```
int degiskenA= 20; // global degisken

void fonk1(){
    degiskenA= 5;

}

int fonk2(){
    int degiskenA= 7;
    return degiskenA;
}


int main() {

    printf("%d\n",degiskenA);   // globali yazdırdık
    fonk2();                    // local degisken
    printf("%d\n",degiskenA);   // local degisken oldugu için etki yok
    printf("%d\n",fonk2());     // local degiskeni yazdırdık
    fonk1();                    // local deegil globali kullandı dikkat
    printf("%d\n",degiskenA);   // Global degisken oldugu için etkilendi

    return 0;

}
```

# HAFTA9 LAB Soruları

Aşağıdaki beş problemi C dilinde çözünüz. Her soruda belirtilen fonksiyonu tanımlayınız ve fonksiyon çağrısını main() içerisinde gerçekleştiriniz. İlk iki soruda normal, son üç soruda recursive fonksiyon kullanınız.

### 1) Bir Sayının Karesini ve karekokü Hesaplama

Konu: Parametre alan ve değer döndüren fonksiyonlar

Klavyeden -1000 ile 1000 arasında bir tam sayı alınız. Bu sayının karesini ve karekoku hesaplamak için sırasıyla **int kare(int sayi)** ve **float karekok(int sayi)** isimli iki ayrı fonksiyon yazınız. Her iki fonksiyon da sonucu **return** ifadesiyle main() fonksiyonuna göndermelidir. Alınan sayının belirtilen aralıkta olup olmadığını kontrol ederek sonuçları ekrana yazdırınız.

```
Örnek giriş:  4
Beklenen çıktı: Kare: 16    karekok: 2
```

### 2) Bir Sayının Pozitif Bölenlerini Yazdırma

Konu: Parametre alan ve değer döndürmeyen fonksiyon

Klavyeden pozitif bir tam sayı alarak bu sayının bütün pozitif bölenlerini küçükten büyüğe sıralı biçimde ekrana yazdıran bir C programı yazınız. Bölenleri belirlemek ve ekrana yazdırmak için **void bolenleriYazdir(int sayi)** isimli fonksiyonu tanımlayınız. Fonksiyonun değer döndürmemesine dikkat ediniz. Pozitif olmayan girişlerde uygun bir uyarı veriniz.

```
Örnek giriş:  12
Beklenen çıktı: Bolenler: 1 2 3 4 6 12
```

### 3) Recursive Fonksiyon ile 1'den N'e Kadar Sayıların Toplamı

(Faktöriyel sorusuna benzer; çarpım yerine toplama mantığı)

Klavyeden girilen $N$ sayısına kadar (1 dahil, N dahil) olan tüm tam sayıların toplamını özyineli (recursive) olarak hesaplayan C programını yazınız.

### 4) Recursive Fonksiyon ile Rakamlar Toplamı

Konu: Sayıyı daha küçük alt problemlere ayırma

Klavyeden negatif olmayan bir tam sayı alınız. Sayının rakamları toplamını hesaplamak için int rakamlarToplami(int n) isimli recursive bir fonksiyon yazınız. Son rakamı belirlemek için % operatörünü, son rakamı sayıdan çıkarmak için tam sayı bölmesini kullanınız. Fonksiyon içerisinde döngü, dizi veya karakter dizisi kullanmayınız. Negatif sayı girişlerinde uyarı gösteriniz.

```
Örnek giriş:  472
Beklenen çıktı: Rakamlar toplami: 13
```

## Çalışma Soruları

**5) Recursive Fonksiyon ile Sayının Basamak Sayısını Bulma** (Rakamlar toplamı sorusuna benzer; basamak ayırma mantığı)

Klavyeden girilen pozitif bir tam sayının kaç basamaklı olduğunu özyineli (recursive) olarak bulan C programını yazınız.

**6) Recursive Fonksiyon ile EBOB (En Büyük Ortak Bölen) Bulma**

(Üs alma sorusuna benzer; iki parametreli ve Öklid algoritması mantığı)

Klavyeden girilen pozitif iki tam sayının En Büyük Ortak Bölenini (EBOB) Öklid algoritmasını kullanarak özyineli (recursive) olarak hesaplayan C programını yazınız.

# HAFTA9 ÇÖZÜMLER

## 1)

```
#include <stdio.h>
#include <math.h> // sqrt için

// karesi ve karekökü hesaplama
/*  sqrt kullanalım */

 int kare(int sayi);                    // prototip
 float karekok(int sayi);               // prototip

int main(void) {

        int a;
        printf("-1000 ile 1000 arasi sayi giriniz\n");
        scanf("%d",&a);

        if(a<-1000||a>1000)
                printf("hatali bir secim %d\n",a);
        else{
                // fonksiyonlari çağıralim
                int kareSonuc=kare(a);
                printf("Karesi: %d\n",kareSonuc);
                // doğrudan printfle çağrı
                printf("Karekoku: %.2f",karekok(a));
        }
        return 0;
}

 int kare(int sayi){
 return sayi*sayi;
 }

 float karekok(int sayi){
        if(sayi<0){
                printf( "Negatif Sayilarin Karekoku yoktur\n");
                return -1.0;
        }
  // return sqrt(sayi);
  return sqrt((float)sayi); //tip dönüşümü
 }
```

### Ekran Çıktısı

```
-1000 ile 1000 arasi sayi giriniz
11
Karesi: 121
Karekoku: 3.32
--------------------------------
```

## 2)

```
#include <stdio.h>

// recursive 1 den girilen sayiya toplama

int toplamiHesapla(int sayi); //prototip
void toplamiHesapla2(int sayi); //prototip

int sonuc;

int main(void) {
	int deger;
	printf("Hesaplanmasi istenen Sayiyi giriniz\n");
	scanf("%d",&deger);

	printf("Sayinin toplami: %d\n", toplamiHesapla(deger));
	toplamiHesapla2(deger);			// voidle
	printf("Sayinin toplami: %d", sonuc);
	return 0;
}

int toplamiHesapla(int sayi){

// taban
if(sayi==1)
	return 1;
// özyineleme

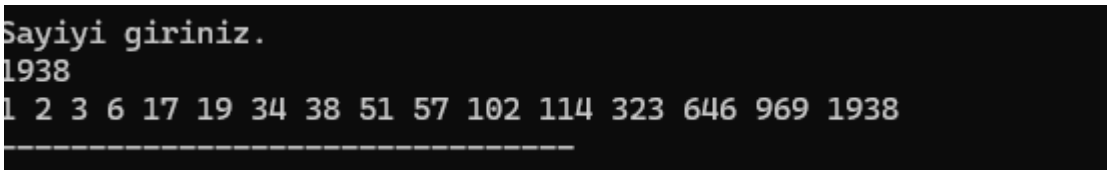
return sayi + toplamiHesapla(sayi-1);

}

void toplamiHesapla2(int sayi){
// taban
if(sayi==0)
	return; // sonlanma için deger dondrurmez
// özyineleme
	sonuc+=sayi;
	toplamiHesapla2(sayi-1);
}
```

**Açıklama:** sayi % i == 0 koşulu, sayının i değerine kalansız bölünüp bölünmediğini sınar. void türündeki fonksiyon sonuç döndürmek yerine ekrana yazdırır.

### Ekran Çıktısı




## 3)

```
#include <stdio.h>
int toplam=0; // global değişken void recursive için lazım

int toplamHesapla(int n);       // fonk prototip
void toplamHesapla2(int n);     // fonk prototip

// recursive fok
int toplamHesapla(int n) {
    // Taban Durumu: 1'e ulaşıldığında durur.
    if (n == 1) {
        return 1;
    }
    // Özyineli Durum: N ile N-1'in toplamını birleştirir.
    return n + toplamHesapla(n - 1);
}


int main() {
    int n;
    printf("Bir N degeri giriniz (N >= 1): ");
    scanf("%d", &n);


    if (n < 1) {
        printf("Gecersiz sayi!\n");
    } else {
        printf("1'den %d'e kadar olan sayilarin toplami: %d\n", n, toplamHesapla(n));
        toplamHesapla2(n);
        printf("1'den %d'e kadar olan sayilarin toplami: %d\n", n, toplam);
    }

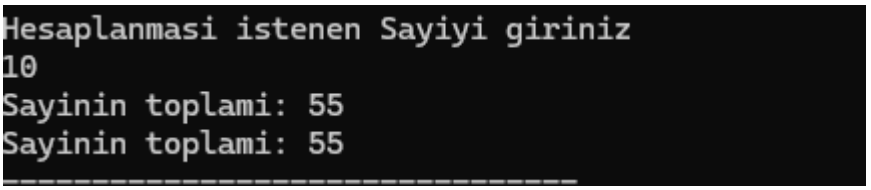
    return 0;
}

// recursive fonk void ile yazsaydık
void toplamHesapla2 (int n){
```

```
        // taban durumu
        if(n==0){
                return; // burdaki return program sonlansın diye değer dondurmüyor zaten void
        }
        // özyineleme

        toplam=toplam + n;
    toplamHesapla2(n-1);
}
```

### Ekran Çıktısı




## 4)

```
#include <stdio.h>

int rakamlarToplami(int n) {
    if (n < 10) {
        return n;
    }
    return (n % 10) + rakamlarToplami(n / 10);
}

int main() {
    int sayi;
    printf("Negatif olmayan bir sayi giriniz: ");
    scanf("%d", &sayi);

    if (sayi < 0) {
        printf("Gecersiz sayi!\n");
    } else {
        printf("Rakamlar toplami: %d\n", rakamlarToplami(sayi));
    }
    return 0;
}
```

**Açıklama:** n % 10 son rakamı bulur; n / 10 ise sayının son rakamını kaldırır. n tek basamaklı olunca recursive çağrılar sona erer.

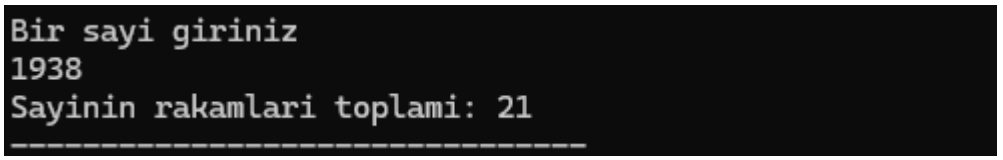
rakamlarToplami(472) için hesaplama:

rakamlarToplami(472) = 2 + rakamlarToplami(47)

= 2 + 7 + rakamlarToplami(4)

= 2 + 7 + 4 = 13

### Ekran Çıktısı




## 5)

```
#include <stdio.h>

int basamakSayisi(int n) {
    // Taban Durumu: Tek basamaklı sayıya ulaşıldığında 1 basamak vardır.
    if (n < 10) {
        return 1;
    }
    // Özyineli Durum: Sayıyı 10'a bölerek bir basamak eksilt ve sayaca 1 ekle
    return 1 + basamakSayisi(n / 10);
}

int main() {
    int sayi;
    printf("Pozitif bir sayi giriniz: ");
    scanf("%d", &sayi);

    if (sayi <= 0) {
        printf("Gecersiz sayi!\n");
    } else {
        printf("Basamak sayisi: %d\n", basamakSayisi(sayi));
    }
    return 0;
}
```

### Ekran Çıktısı

## 6)

```
#include <stdio.h>

int ebob(int a, int b) {
    // Taban Durumu: İkinci sayı 0 olduğunda EBOB ilk sayıdır.
    if (b == 0) {
        return a;
```

```
    }
    // Özyineli Durum: Öklid Algoritması (a yerine b, b yerine a % b yazılır)
    return ebob(b, a % b);
}

int main() {
    int sayi1, sayi2;
    printf("Iki pozitif sayi giriniz: ");
    scanf("%d %d", &sayi1, &sayi2);

    if (sayi1 <= 0 || sayi2 <= 0) {
        printf("Gecersiz sayilar!\n");
    } else {
        printf("EBOB(%d, %d) = %d\n", sayi1, sayi2, ebob(sayi1, sayi2));
    }
    return 0;
}
```

### Ekran Çıktısı